



UNIVERSIDAD POLITÉCNICA DE VICTORIA

**REPORTE TÉCNICO DE ACTIVIDADES DE ESTADÍA
UNIVERSITARIA**

EN HONNE S.A. DE C.V.

QUE PRESENTA

EDER OMAR ZÚÑIGA ZAVALA

**EN CUMPLIMIENTO DE LA ESTADÍA TSU EN
DESARROLLO DE SOFTWARE MULTIPLATAFORMA**

**ASESOR INSTITUCIONAL
DR. SAID POLANCO MARTAGÓN**

**UNIDAD ECONÓMICA RECEPTORA:
HONNE S.A. DE C.V.**

**ASESOR DE LA UNIDAD ECONÓMICA
OSCAR DAVID GALINDO OLVERA**

Ciudad Victoria, Tamaulipas, Agosto de 2026.



Formato de autorización para exposición de Estadía de Técnico Superior Universitario

Fecha y lugar en que se emite: **Ciudad Victoria, Tamaulipas, a 31 de Julio de 2026**

Nombre completo del estudiante: **Eder Omar Zúñiga Zavala**

Matrícula: **2430206**

Programa Académico: **Ingeniería en Tecnologías de la Información e Innovación Digital**

Por medio del presente, se le informa que tras realizar su Estadía en la empresa **Honne S.A. de C.V.**, desarrollando el **Reporte Técnico de Actividades de Estadía Universitaria**, durante el periodo comprendido **Mayo-Agosto 2026** logrando una ponderación de ____ (60 % posibles) para el criterio de reporte; por lo tanto, se otorga la oportunidad de exponer el trabajo realizado por medio de un informe ejecutivo programado para el día **12** del mes de **Agosto** del año en curso en un horario de **10:00 hrs** en modalidad **Presencial** en la **Oficina I-XXX69**.

Sin otro particular, se le recomienda estar disponible para iniciar su exposición 10 minutos antes de la indicación oficial.

Atentamente

Dr. Said Polanco Martagón
Asesor institucional



Control sumativo de Estadía para Técnico Superior Universitario

Criterio	Valor / Ponderación	Calificación
Reporte	60 %	
Exposición	40 %	
Calificación final		

Datos de la evaluación sumativa

Fecha: **12 / Agosto / 2026**

Dr. Said Polanco Martagón
Asesor institucional

Agradecimientos

Se agradece a Honne S.A. de C.V. por la oportunidad de realizar esta Estadía y por el apoyo brindado durante su desarrollo. De manera particular, se agradece al asesor de la unidad económica, Oscar David Galindo Olvera, por su guía y disposición a lo largo de las actividades realizadas, así como al asesor institucional, Dr. Said Polanco Martagón, por su acompañamiento y retroalimentación en la elaboración de este reporte.

Resumen

El presente reporte documenta las actividades realizadas durante la Estadía Universitaria en Honne S.A. de C.V., empresa de transformación tecnológica y consultoría cloud, en su Centro de Operación y Excelencia Multicloud (CCoE) ubicado en calle 533 Maratines, Ciudad Victoria, Tamaulipas. La Estadía se desarrolló en el área de Mesa de Servicio del CCoE, cuyo objetivo principal fue apoyar en el uso de las distintas herramientas de monitoreo del área y en el desarrollo de sus actividades diarias, contribuyendo a distribuir la carga de trabajo del equipo.

Para ello, se verificaron alertas activas de AWS mediante Amazon CloudWatch, aplicando un umbral de persistencia para distinguir falsos positivos de incidentes reales; se revisaron logs de instancias en Google Cloud Platform bajo criterios de errores y latencia; se dio seguimiento a las soluciones de TIBCO Scribe, inicialmente de forma manual y posteriormente mediante un script en Python desarrollado por iniciativa propia, que automatizó la detección de fallas y su notificación a través de un bot de Telegram; y se dio seguimiento constante a las alertas de ServiceNow. Para el desarrollo del script fue necesario aplicar conocimientos de la materia de Redes, además de técnicas de ingeniería inversa de APIs, dado que la plataforma utilizada no contaba con documentación pública para dicho fin.

Como resultado, las alertas de AWS acumuladas se clasificaron por completo, y el script de monitoreo de TIBCO Scribe logró automatizar una revisión que antes tomaba minutos de trabajo manual varias veces al día, liberando aproximadamente 19 horas mensuales de trabajo del equipo. Se concluye que los objetivos planteados se cumplieron, destacando el desarrollo del script como la aportación más significativa de la Estadía, mientras que la automatización de las demás herramientas de monitoreo queda como una oportunidad de mejora a futuro.

Abstract

This report documents the activities carried out during the University Internship (Estadía) at Honne S.A. de C.V., a technology transformation and cloud consulting firm, at its Cloud Operations and Multicloud Excellence Center (CCoE) located at calle 533 Maratines, Ciudad Victoria, Tamaulipas. The internship took place in the CCoE's Service Desk area, whose main objective was to support the use of the area's various monitoring tools and its daily activities, helping to distribute the team's workload.

To this end, active AWS alerts were verified using Amazon CloudWatch, applying a persistence threshold to distinguish false positives from real incidents; logs of instances in Google Cloud Platform were reviewed under error and latency criteria; TIBCO Scribe solutions were monitored, initially by hand and later through a Python script developed on personal initiative, which automated failure detection and notification through a Telegram bot; and ServiceNow alerts were monitored continuously. Developing the script required applying knowledge from the Networking course, as well as API reverse-engineering techniques, since the platform used did not have public documentation for that purpose.

As a result, the accumulated AWS alerts were fully classified, and the TIBCO Scribe monitoring script automated a review that previously took minutes of manual work several times a day, freeing up approximately 19 hours of team work per month. It is concluded that the proposed objectives were met, with the development of the script standing out as the most significant contribution of the internship, while automating the remaining monitoring tools remains an opportunity for future improvement.

Contenido temático

Agradecimientos	III
Resumen	IV
Abstract	V
1 Introducción	1
2 Capítulo 1 — Generalidades de la empresa	2
2.1 Antecedentes históricos	2
2.2 Giro y actividad principal, productos o servicios	2
2.3 Misión, visión, valores y política de calidad	3
2.4 Organigrama y ubicación del departamento	3
2.5 Descripción del área de trabajo y sus funciones	4
3 Capítulo 2 — Planteamiento del problema	6
3.1 Antecedentes del problema	6
3.2 Definición del problema	6
3.3 Justificación	7
3.4 Objetivos	7
3.4.1 Objetivo general	7
3.4.2 Objetivos específicos	7
3.5 Alcance y limitaciones	8
4 Capítulo 3 — Marco teórico	9
5 Capítulo 4 — Metodología / desarrollo	10
5.1 Diagnóstico inicial	10
5.2 Plan de trabajo	10
5.3 Desarrollo por etapas	11
5.4 Implementación	14
6 Capítulo 5 — Resultados	16
6.1 Estado final	16
6.2 Comparativo antes/después	16
6.3 Evidencia fotográfica	16

6.4	Análisis de los resultados	16
7	Conclusiones y recomendaciones	19
7.1	Cierre por objetivo específico	19
7.2	Aprendizaje profesional y personal	19
7.3	Recomendaciones a la empresa	20
7.4	Recomendaciones al programa académico	20
7.5	Trabajo pendiente o siguiente fase	20
	Bibliografía	21
	Anexos	22

1. Introducción

Honne S.A. de C.V. es una firma de transformación tecnológica y consultoría cloud, con su Centro de Operación y Excelencia Multicloud (CCoE) ubicado en Ciudad Victoria, Tamaulipas. La Estadía se realizó en la Mesa de Servicio de dicho centro, área encargada de atender las solicitudes de los clientes relacionadas con sus instancias en la nube (AWS, Google Cloud Platform y Microsoft Azure), así como de identificar y reportar errores o fallas en dichas instancias.

Al iniciar la Estadía, la Mesa de Servicio manejaba varios frentes de trabajo de forma simultánea, entre ellos un número considerable de alertas activas generadas en AWS que aún no habían sido verificadas, ya que esta tarea era responsabilidad de todo el equipo sin que nadie la tuviera asignada de forma específica. Como estas alertas podían corresponder tanto a fallas reales como a falsos positivos, y la empresa tiene la obligación contractual de reportar los incidentes reales a sus clientes, era necesario distribuir esta carga de trabajo; en esa tarea participé como practicante, apoyando al equipo.

Durante la Estadía se llevó a cabo la verificación de alertas de AWS para identificar falsos positivos, la revisión de logs de instancias en Google Cloud Platform, el monitoreo de las soluciones de TIBCO Scribe y el desarrollo, por iniciativa propia, de un script que automatizó parte de ese monitoreo mediante notificaciones a un bot de Telegram, además del seguimiento a las alertas registradas en ServiceNow. Estas actividades permitieron distribuir la carga de trabajo del área y agilizar la atención de sus actividades diarias.

2. Capítulo 1 — Generalidades de la empresa

2.1. Antecedentes históricos

Honne es una empresa especializada en transformación tecnológica, consultoría y gestión de infraestructura de TI que cuenta con más de 8 años de experiencia en el mercado. A lo largo de su trayectoria ha consolidado su operación a nivel internacional, habiendo entregado más de 300 proyectos [1].

Actualmente cuenta con presencia operativa en 6 países: México, Colombia, Chile, Estados Unidos, Francia y España. Su principal Centro de Operación y Excelencia Multicloud (CCoE), donde se operan servicios multicloud, se ubica en Ciudad Victoria, Tamaulipas, sumado a su oficina corporativa en Monterrey y sedes en la Ciudad de México, Bogotá, Santiago, Houston, París y Madrid. Asimismo, ha forjado alianzas de alto nivel como partner de AWS, Google Cloud Platform, Microsoft Azure, Alibaba Cloud, Oracle y NVIDIA [1].

2.2. Giro y actividad principal, productos o servicios

Según la información institucional de la empresa [1], su giro y actividad principal, así como sus productos y servicios, se describen a continuación.

Giro. Honne es una firma de ingeniería de valor empresarial (*Engineering Enterprise Value*), enfocada en el sector de las Tecnologías de la Información, la consultoría cloud y la gestión de infraestructura.

Actividad principal. La empresa se dedica a combinar el uso de la nube pública, la Inteligencia Artificial (IA) y las operaciones de TI para alinear la tecnología con los objetivos de negocio de sus clientes, generando un impacto medible y apoyándolos a escalar su infraestructura y operar con estabilidad.

Productos o servicios. Sus soluciones se enmarcan en el modelo *Enterprise Value Framework*, estructurado en las siguientes dimensiones:

- **Stability (Estabilidad):** gestión y monitoreo continuo, mediante plataformas como Google Cloud o AWS, para garantizar una operación estable, segura y controlada que asegure la eficiencia y continuidad del negocio.

- **Modernization (Modernización):** transformación estructural y evolución de la arquitectura tecnológica de los clientes para que puedan crecer con agilidad y sin fricción.
- **Intelligence (Inteligencia):** aplicación de análisis de datos e Inteligencia Artificial para optimizar las operaciones corporativas y fomentar la toma de mejores decisiones.
- **AI Fusion:** práctica dedicada a implementar la IA, llevando modelos desde la fase de piloto hasta su producción real con impacto en el negocio.

2.3. Misión, visión, valores y política de calidad

De acuerdo con la información institucional de la empresa [1], la misión de Honne es crear valor para sus clientes mediante la innovación y las Tecnologías de la Información, mientras que su visión es ser líderes mundiales en la gestión de nube pública y en la creación de servicios de próxima generación.

Sus valores fundamentales son:

- Confiabilidad
- Servicio excepcional
- Innovación
- Mejora continua
- Máxima productividad

En cuanto a su política de calidad, Honne opera bajo un ecosistema de estándares y certificaciones internacionales —ISO 9001 (Gestión de la Calidad), ISO 20000 (Gestión de Servicios de TI), ISO 27001 (Seguridad de la Información), ITIL, CMMI y TOGAF— que garantizan la excelencia operativa en cada entrega. Su compromiso operativo se rige, además, por la filosofía de trabajo *One Team*, que fomenta equipos integrados bajo una sola visión, desde la consultoría inicial hasta el soporte y el monitoreo continuo.

2.4. Organigrama y ubicación del departamento

El Centro de Operación y Excelencia Multicloud (CCoE), donde se realizó la Estadía, se ubica en calle 533 Maratines, Ciudad Victoria, Tamaulipas (ver Figura 1).



Figura 1: Ubicación del CCoE de Honne en Ciudad Victoria, Tamaulipas.

La Figura 2 muestra la estructura organizacional de Honne. La Estadía se realizó dentro del equipo de Mesa de Servicio, ubicado en la columna de Servicios Administrados 2.0.

2.5. Descripción del área de trabajo y sus funciones

El área en la que se realizó la Estadía es la Mesa de Servicio del CCoE. Su función principal es atender las solicitudes de los clientes relacionadas con sus instancias en la nube (AWS, Google Cloud Platform y Microsoft Azure), bases de datos y demás recursos de infraestructura, así como identificar errores o fallas en dichas instancias y reportarlos a los usuarios para que soliciten su corrección.

La operación del área combina distintos canales y herramientas de trabajo. El correo electrónico es el canal principal para recibir alertas de errores e incidencias generadas por los distintos proveedores de cómputo en la nube. Además, para clientes específicos se emplean plataformas de monitoreo dedicadas como ServiceNow y TIBCO Scribe, mientras que Zendesk se utiliza principalmente para la gestión y asignación de tickets a las áreas responsables de la corrección de errores, como SysAdmin.

Los tickets atendidos abarcan una variedad de solicitudes de infraestructura y soporte, tales

img/figura_organigrama.png

Figura 2: Organigrama de Honne, con la Mesa de Servicio dentro de Servicios Administrados 2.0.

como la activación de autenticación multifactor (MFA), el restablecimiento de contraseñas, el ajuste de husos horarios, la validación de respaldos, la baja de usuarios, la asistencia en sesiones de trabajo y la elaboración de reportes periódicos de facturación, entre otros.

Como apoyo adicional, el área utiliza bots que automatizan parte del monitoreo, enviando notificaciones a través de Telegram sobre el estado de los servidores y la llegada de nuevos tickets en Zendesk.

3. Capítulo 2 — Planteamiento del problema

3.1. Antecedentes del problema

Como ya se describió en el capítulo anterior, la Mesa de Servicio del CCoE manejaba varios frentes de trabajo de forma simultánea: monitoreo de correos, seguimiento de páginas de clientes específicos y gestión de tickets en Zendesk. A esta carga se sumaba un número considerable de alertas activas generadas en AWS que aún no habían sido verificadas, ya que el proceso para revisarlas no estaba asignado a una persona en específico.

3.2. Definición del problema

La saturación de la Mesa de Servicio se reflejaba de forma concreta en el número de alertas activas sin verificar: se llegaron a registrar 129 alertas generadas en AWS pendientes de revisión (ver Figura 3).

	C	D	E	F
109	honne_transpals	AWS_EC2 Transpals AD Hermes betterez Uso CPU i-01f02d8d8df9bc127	sector_privado	us-west-2
110	honne_transpals	AWS_EC2 Transpals AD Hermes betterez Uso Memoria i-01f02d8d8df9bc127	sector_privado	us-west-2
111	honne_transpals	AWS_EC2 Transpals I-GOAL-APP-TTI-PROD - Uso Disco C i-093662a9976cd3622	sector_privado	us-west-2
112	honne_transpals	AWS_EC2 Transpals I-GOAL-APP-TTI-PROD - Uso Memoria i-093662a9976cd3622	sector_privado	us-west-2
113	honne_transpals	AWS_EC2 Transpals I-GOAL-APP-TTI-PROD Disponibilidad Instancia i-093662a9976cd3622	sector_privado	us-west-2
114	honne_transpals	AWS_EC2 Transpals I-GOAL-APP-TTI-PROD Uso CPU i-093662a9976cd3622	sector_privado	us-west-2
115	honne_transpals	AWS_EC2 Transpals I-GOAL-APP-TTI-PROD - Uso Disco i-0bb502ee65e337214	sector_privado	us-west-2
116	honne_transpals	AWS_EC2 Transpals I-GOAL-MAPS-TTI-PROD - Uso Memoria i-0bb502ee65e337214	sector_privado	us-west-2
117	honne_transpals	AWS_EC2 Transpals I-GOAL-MAPS-TTI-PROD Disponibilidad Instancia i-0bb502ee65e337214	sector_privado	us-west-2
118	honne_transpals	AWS_EC2 Transpals I-GOAL-MAPS-TTI-PROD Uso CPU i-0bb502ee65e337214	sector_privado	us-west-2
119	honne_transpals	AWS_VPN Transpals VPN-EAXGRID vpn-04f02c8292ec45d63	sector_privado	us-west-2
120	honne_transpals	awssec2-i-093662a9976cd3622-GreaterThanOrEqualToThreshold-StatusCheckFailed	sector_privado	us-west-2
121	honne_transpals	awssec2-i-093662a9976cd3622-GreaterThanOrEqualToThreshold-StatusCheckFailed_System	sector_privado	us-west-2
122	honne_umm	AWS_EC2 UMM CONTRAQ-UMM-R4 Disponibilidad Instancia i-0aa3846d65fb22826	sector_publico	us-east-1
123	honne_umm	AWS_EC2 UMM CONTRAQ-UMM-R4 Espacio Libre Disco C i-0aa3846d65fb22826	sector_publico	us-east-1
124	honne_umm	AWS_EC2 UMM CONTRAQ-UMM-R4 Uso CPU i-0aa3846d65fb22826	sector_publico	us-east-1
125	honne_umm	AWS_EC2 UMM CONTRAQ-UMM-R4 Uso Memoria i-0aa3846d65fb22826	sector_publico	us-east-1
126	honne_universidad_ins	AWS_EC2 Universidad Insurgentes Moodle-QA Disponibilidad Instancia i-01972af940aaf159d	sector_publico	us-east-2
127	honne_universidad_ins	AWS_EC2 Universidad Insurgentes Moodle-QA Uso Partición / i-01972af940aaf159d	sector_publico	us-east-2
128	honne_universidad_ins	AWS_EC2 Universidad Insurgentes Moodle-QA Uso CPU i-01972af940aaf159d	sector_publico	us-east-2
129	honne_universidad_ins	AWS_EC2 Universidad Insurgentes Moodle-QA Uso Memoria i-01972af940aaf159d	sector_publico	us-east-2

Figura 3: Registro interno de alertas activas de AWS pendientes de verificación.

Cada alerta debía revisarse manualmente para descartar falsos positivos, generados principalmente por dos causas: instancias que se apagaban de forma programada, lo cual generaba una alerta como si hubiera fallado aunque en realidad estaba funcionando como se esperaba; e instancias que ejecutaban tareas programadas, como respaldos, en horarios específicos, lo cual elevaba temporalmente el uso de CPU, disco, memoria o particiones hasta el 100 % y disparaba alertas de sobrecarga.

Aunque la verificación de alertas era responsabilidad de toda la Mesa de Servicio, no había una persona dedicada específicamente a esta tarea, por lo que las alertas reales y los falsos positivos se

acumulaban en la misma bandeja, dificultando distinguir con rapidez cuáles requerían atención inmediata.

3.3. Justificación

Verificar cada alerta antes de reportarla era necesario porque Honne tiene la obligación contractual de informar a sus clientes sobre incidentes reales en sus instancias. Si un incidente real no se reportaba a tiempo, el cliente podía detectarlo por su cuenta y presentar una queja, lo cual representaba un riesgo de incumplimiento de contrato para la empresa. Al mismo tiempo, descartar los falsos positivos evitaba generar reportes innecesarios hacia clientes que no correspondían a una falla real.

Además, la incorporación de practicantes a esta tarea tenía como propósito liberar parte de la carga de trabajo de la Mesa de Servicio, distribuyendo la verificación de alertas para que el equipo pudiera enfocarse en la atención de incidentes reales y otras solicitudes.

3.4. Objetivos

3.4.1. Objetivo general

Integrarse a la Mesa de Servicio del CCoE para apoyar en el uso de sus distintas herramientas de monitoreo y en el desarrollo de sus actividades diarias, contribuyendo a distribuir la carga de trabajo del área.

3.4.2. Objetivos específicos

- Verificar las alertas activas generadas en AWS para identificar falsos positivos y reportar los incidentes reales a los clientes correspondientes.
- Revisar los logs de instancias en Google Cloud Platform para detectar problemas de latencia y errores recurrentes.
- Monitorear el estado de las soluciones y Agents de TIBCO Scribe (Scribesoft) para verificar su disponibilidad.
- Desarrollar un script de monitoreo automatizado que notificara mediante un bot de Telegram cuando una solución de TIBCO Scribe presentara fallas recurrentes.

- Dar seguimiento a las alertas registradas en ServiceNow para notificar oportunamente al equipo responsable de su atención.

3.5. Alcance y limitaciones

El apoyo brindado se centró en las herramientas de monitoreo utilizadas por la Mesa de Servicio del CCoE: alertas de AWS, logs de Google Cloud Platform, soluciones de TIBCO Scribe y alertas de ServiceNow, además del desarrollo de un script de automatización para una de ellas.

Como limitación, la función en todos los casos fue de detección y notificación, no de corrección directa de las fallas, ya que la resolución de incidentes correspondía a otras áreas, como SysAdmin. Tampoco se contó con un diagnóstico formal previo: las actividades fueron asignadas progresivamente por el asesor empresarial conforme surgían necesidades en el área.

4. Capítulo 3 — Marco teórico

En el monitoreo de infraestructura en la nube, se conoce como falso positivo a una alerta que se genera por un comportamiento aparentemente anómalo, pero que en realidad corresponde a una operación normal o programada del sistema. Cuando estas alertas no se filtran, se produce lo que se conoce como fatiga de alertas: el equipo revisa notificaciones innecesarias, lo que puede provocar que un incidente real pase desapercibido entre el ruido [2].

Para distinguir un falso positivo de un incidente real, es común aplicar un umbral de persistencia: un criterio de tiempo o de repeticiones que debe cumplirse antes de considerar que un comportamiento anómalo representa una falla real. Durante la Estadía, este criterio se aplicó de forma práctica en la verificación de alertas de AWS: si el uso elevado de un recurso (CPU, disco o memoria) se mantenía por más de 15 minutos, se consideraba un incidente real y se reportaba; si el comportamiento coincidía con un patrón programado y recurrente, se descartaba como falso positivo.

Honne opera bajo el marco de gestión de servicios de TI ITIL [1], cuyo proceso de gestión de incidentes establece que un evento debe verificarse y clasificarse antes de ser reportado o escalado [3]. La práctica de verificación descrita en este reporte es consistente con ese proceso, aunque no se aplicó siguiendo una capacitación formal en el marco.

Otro concepto relevante es la automatización en monitoreo, que consiste en el uso de herramientas y programas para supervisar sistemas de forma continua y notificar de manera automática al equipo responsable cuando se detecta una condición anómala [4]. El script desarrollado durante la Estadía para monitorear TIBCO Scribe, descrito en el Capítulo 4, aplica este concepto al conectarse periódicamente a la API de la herramienta y enviar notificaciones a un bot de Telegram cuando detecta fallas recurrentes.

Como TIBCO Scribe no ofrece una API pública documentada para consultar el historial de ejecuciones, fue necesario recurrir a la ingeniería inversa de APIs: el proceso de analizar el comportamiento, los endpoints y las estructuras de datos de una API sin depender de documentación oficial, inspeccionando el tráfico de red que genera la aplicación para identificar y replicar sus llamadas internas [5]. Esta técnica se describe con más detalle en el Capítulo 4.

5. Capítulo 4 — Metodología / desarrollo

5.1. Diagnóstico inicial

El estado inicial de la verificación de alertas de AWS ya se describió en el Capítulo 2: existía un número considerable de alertas activas sin verificar, sin que esa tarea estuviera asignada a una persona en específico.

Para TIBCO Scribe no existía un número exacto de fallas acumuladas, pero sí una carga considerable de trabajo: el monitoreo cubría a dos instituciones educativas clientes de Honne, cada una con alrededor de 16 pestañas de registros (cerca de 32 en total) donde se reportaba si las soluciones, instancias y Agents del cliente estaban funcionando correctamente. Revisar todo esto de forma manual, una por una, era la única forma de detectar fallas.

5.2. Plan de trabajo

La Estadía comenzó a mediados de mayo de 2026. Durante las primeras semanas, el trabajo se realizó completamente en modalidad remota, ya que las oficinas de Honne se encontraban en remodelación. A partir de julio, una vez trasladados a las oficinas físicas, se adoptó una modalidad híbrida: home office los lunes y viernes, y trabajo presencial de martes a jueves. Dentro del equipo existía además una rotación de actividades: a cada integrante le correspondían una o dos actividades principales en un momento dado, además de subtarefas de apoyo.

En términos generales, las actividades se distribuyeron de la siguiente manera:

- Primer mes: verificación de alertas activas de AWS.
- Segundo mes: revisión de logs de instancias en Google Cloud Platform.
- Tercer mes en adelante: monitoreo de TIBCO Scribe y desarrollo del script de automatización.

El seguimiento a las alertas de ServiceNow se mantuvo como una tarea constante a lo largo de toda la Estadía, sin estar limitado a una etapa específica.

5.3. Desarrollo por etapas

Verificación de alertas de AWS. Esta etapa ya se describió a detalle en el Capítulo 2: se monitoreaban las alertas activas en la consola de AWS, apoyándose en las métricas de Amazon CloudWatch para distinguir un falso positivo de una falla real, aplicando el umbral de persistencia de 15 minutos descrito en el Marco teórico. La Figura ?? muestra un ejemplo de patrón de estrés identificado en las gráficas de CloudWatch.



Figura 4: Gráfica de Amazon CloudWatch mostrando un patrón de estrés recurrente.

Revisión de logs en Google Cloud Platform. Se revisaban los registros de las instancias del cliente en la consola de Google Cloud Platform, aplicando dos criterios: si se acumulaban más de 20 registros de error, se consideraba una posible caída del sistema; y si el nivel de latencia de los logs superaba el valor 20 y se mantenía así de forma persistente, también se reportaba como incidente. Este segundo criterio es, de nuevo, un umbral de persistencia: no bastaba con

un registro elevado aislado, la condición debía sostenerse en el tiempo (ver Figura 4). El uso de los recursos primarios de la instancia también se supervisaba de forma gráfica (ver Figura 5).

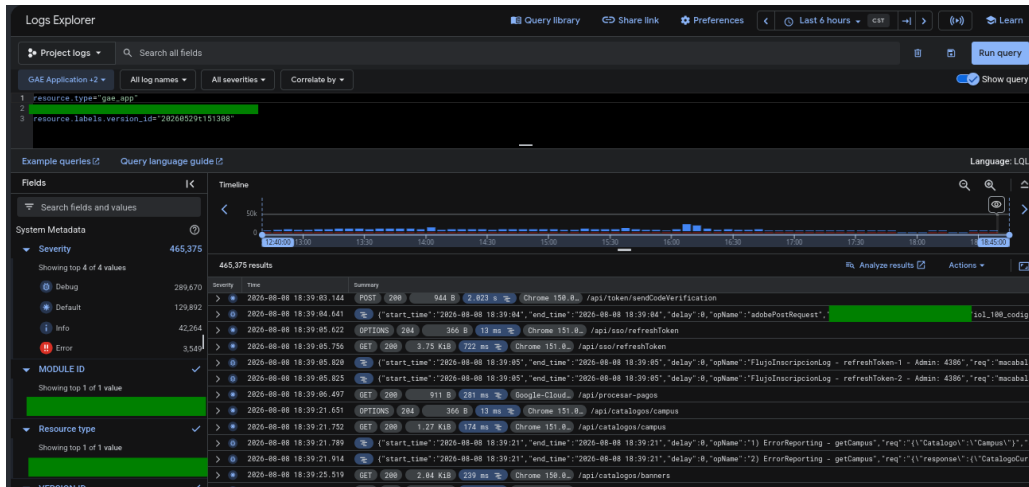


Figura 5: Registros de error en la consola de Google Cloud Platform.

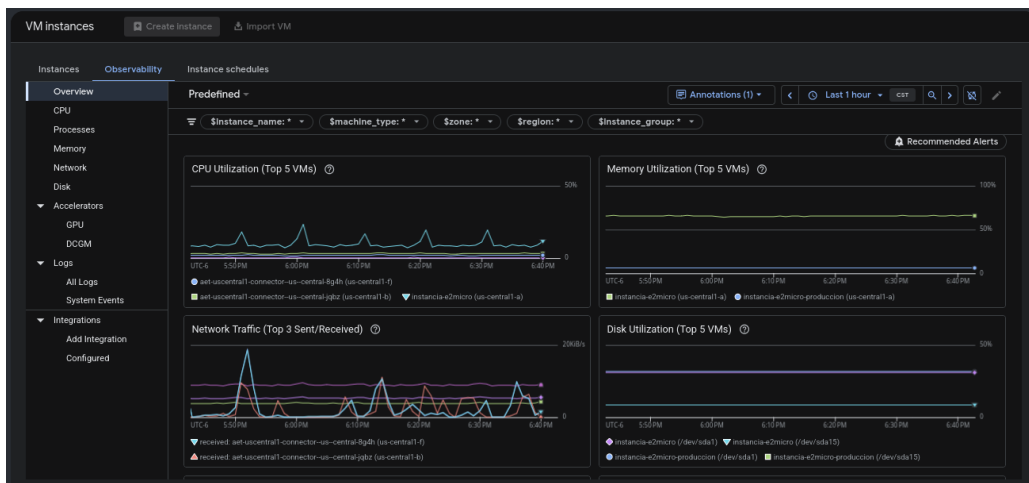


Figura 6: Gráfica de uso de recursos primarios de la instancia en Google Cloud Platform.

Monitoreo de TIBCO Scribe (Scribesoft). Antes de automatizarlo, esta revisión se hacía completamente a mano: en una pestaña con el listado de Agents/instancias de cada cliente, había que seleccionar cada uno, aplicar el filtro y esperar el resultado de la consulta, repitiendo el proceso más de 30 veces por revisión. Cada solución podía mostrar cuatro estados: completado exitosamente (verde), completado con errores (amarillo), en proceso (gris) o error fatal (rojo). Las reglas para considerar una falla real no las definí yo: las obtuve consultando con el personal del área, que ya las aplicaba de forma empírica en su trabajo diario. Quedaron así: más de 10 lecturas consecutivas en gris y más de 10 minutos transcurridos; más de 10 lecturas consecutivas en amarillo, sin importar el tiempo; o una sola lectura en rojo, que se reportaba de inmediato.

La Figura ?? muestra la pestaña de Agents/instancias que se revisaba manualmente, antes de automatizar el proceso.



Figura 7: Pestaña de Agents/instancias de TIBCO Scribe revisada manualmente antes de la automatización.

Desarrollo del script de monitoreo automatizado. Ante lo repetitivo del proceso anterior, identifiqué la oportunidad y desarrollé, por iniciativa propia, un script en Python que automatiza esta revisión, aplicando las mismas reglas de conteo y tiempo obtenidas del equipo, y notificando al equipo mediante un bot de Telegram solo cuando una racha cumple la regla, sin repetir el aviso si ya había sido reportada.

El acceso a la plataforma ya existía previamente: Honne cuenta con cuenta activa en Scribesoft como parte de sus herramientas de integración, y usé mi acceso ya autorizado, sin gestionar credenciales nuevas. Sin embargo, Scribesoft no ofrece una API pública documentada para

consultar el historial de ejecuciones de forma automatizada. Para resolverlo, inspeccioné el tráfico de red del panel web de Scribesoft con las herramientas de desarrollador del navegador, identifiqué las llamadas que la página hace a su servidor para obtener el historial de cada solución, y repliqué esas mismas llamadas desde el script en Python, autenticándome con las mismas credenciales que ya usaba para entrar por la interfaz. El script consulta esta información cada 2 minutos para todas las soluciones de ambos clientes. En esta primera versión, la regla del estado rojo (error fatal) todavía no quedaba automatizada y se seguía revisando de forma manual.

La Figura 6 resume la lógica que sigue la versión final del script en cada revisión (con el intervalo de 4 minutos y las mejoras descritas en el Capítulo 5).

El siguiente fragmento, tomado de la versión final del script, muestra la función que decide si una racha de estados debe reportarse, aplicando las reglas de conteo y tiempo descritas arriba:

Listing 1: Función que decide si reportar una alerta, según la regla configurada para cada estado.

```
1 def debe_reportar(estado, inicio, cantidad_en_racha):
2     regla = REGLAS.get(estado)
3
4     if regla is None:
5         return False
6
7     if regla["tipo"] == "conteo":
8         return cantidad_en_racha >= regla["limite"]
9
10    if regla["tipo"] == "tiempo":
11        ahora = datetime.now(timezone.utc)
12        minutos_transcurridos = (ahora - inicio).total_seconds() / 60
13        return minutos_transcurridos >= regla["limite_min"]
14
15    if regla["tipo"] == "conteo_y_tiempo":
16        ahora = datetime.now(timezone.utc)
17        minutos_transcurridos = (ahora - inicio).total_seconds() / 60
18        cumple_conteo = cantidad_en_racha >= regla["limite"]
19        cumple_tiempo = minutos_transcurridos >= regla["limite_min"]
20        return cumple_conteo and cumple_tiempo
21
```

```
return False
```

Seguimiento a ServiceNow. A diferencia de las otras herramientas, ServiceNow no generaba notificaciones automáticas de error, por lo que había que mantenerse atento a la bandeja de forma constante mientras se realizaban otras tareas, dado que correspondía a un cliente importante (ver Figura ??). La frecuencia de revisión variaba según la herramienta: GCP se revisaba cada hora, TIBCO Scribe cada 30 minutos, y ServiceNow de forma continua. Hacia el final de la Estadía, un compañero del equipo desarrolló un bot independiente que revisaba ServiceNow cada 12 segundos y notificaba automáticamente los errores; este desarrollo no formó parte de mi trabajo.

De la formación académica, la materia de Redes fue la más aplicada en estas actividades, al tener que aprender a utilizar distintos tipos de *Software as a Service* y familiarizarme con el panel de AWS y CloudWatch.

5.4. Implementación

El script de monitoreo de TIBCO Scribe se implementó y se ejecutó de forma local en mi propio equipo de cómputo, reportando los errores detectados bajo las condiciones descritas en la etapa anterior durante el resto de la Estadía.

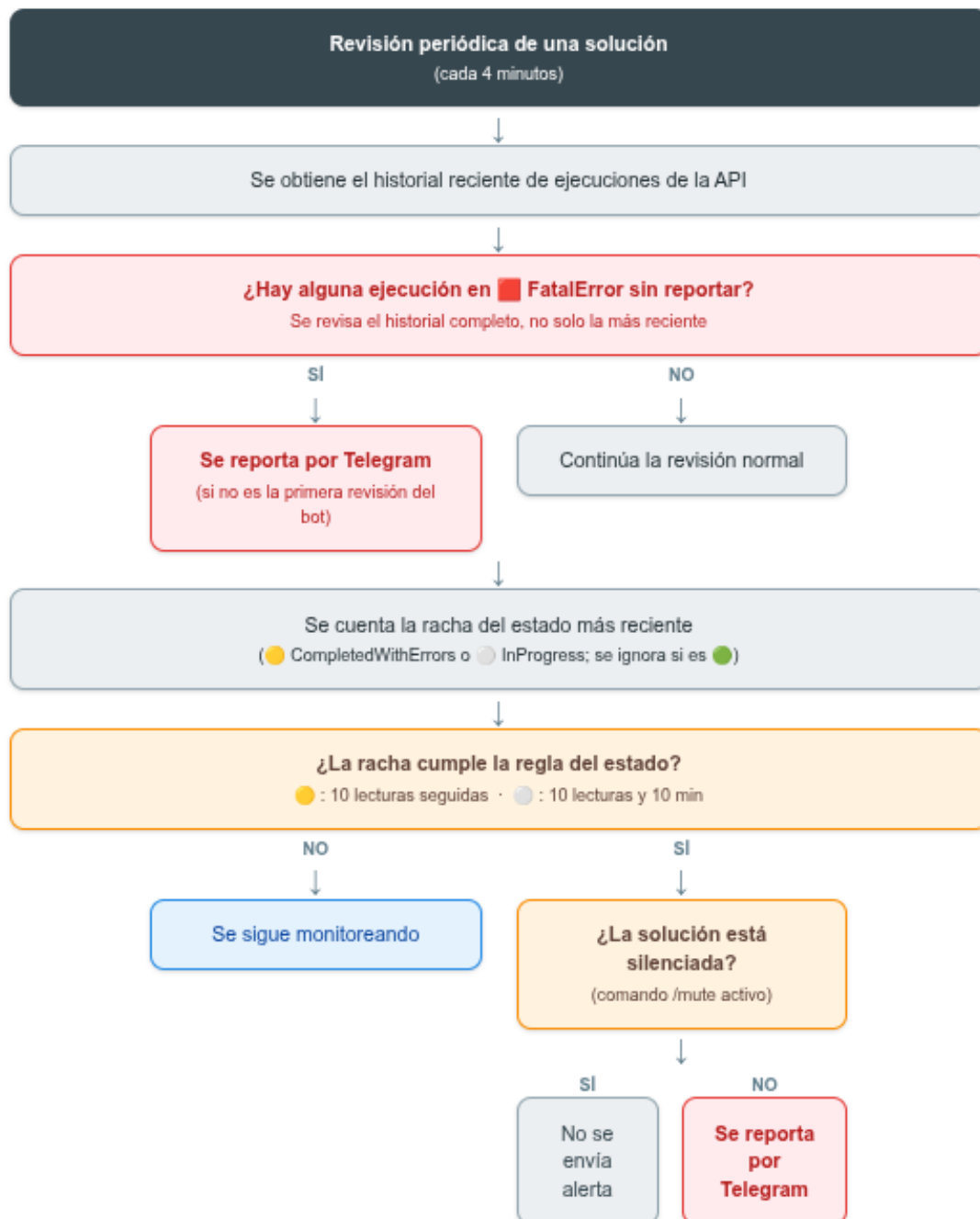


Figura 8: Diagrama de flujo de la lógica del bot de monitoreo de TIBCO Scribe.

img/figura_servicenow.png

Figura 9: Bandeja de alertas de ServiceNow revisada durante la Estadía.

6. Capítulo 5 — Resultados

6.1. Estado final

Al cierre de la Estadía, las 129 alertas de AWS registradas en el diagnóstico inicial se clasificaron por completo entre patrones programados y fallas reales; ese trabajo tomó varios días de revisión continua.

El script de monitoreo de TIBCO Scribe también evolucionó respecto a la primera versión descrita en el Capítulo 4. En la versión final, la regla del estado de error fatal —que en un inicio se seguía revisando de forma manual— quedó automatizada: el script revisa el historial completo de cada solución, reporta cada ejecución en error fatal de forma individual sin repetir el mismo evento dos veces, y evita alertar sobre caídas que ya existían antes de que el script arrancara, para no saturar de avisos viejos. También se agregó la posibilidad de silenciar temporalmente, mediante un comando enviado al bot de Telegram, las alertas de una solución específica cuando el equipo ya tenía conocimiento de una falla en curso.

6.2. Comparativo antes/después

Antes de contar con el script, revisar manualmente el estado de las soluciones de TIBCO Scribe tomaba entre 3 y 4 minutos por cada verificación, ya que era necesario abrir cada pestaña de forma individual, aplicar el filtro y esperar el resultado de la consulta. Esta revisión se repetía cada 30 minutos a lo largo de la jornada laboral. Con el script en funcionamiento, esa verificación se realiza de forma automática cada pocos minutos, sin requerir tiempo de ninguna persona del equipo; el trabajo humano se limita a atender las notificaciones que el bot envía cuando detecta una falla real.

6.3. Evidencia fotográfica

6.4. Análisis de los resultados

El objetivo de automatizar el monitoreo de TIBCO Scribe se cumplió por completo: la versión final del script cubre incluso la regla del estado de error fatal, que en un inicio se seguía revisando de forma manual. Suponiendo una jornada laboral de 8 horas y una revisión manual cada 30 minutos (16 revisiones al día, de 3 a 4 minutos cada una), esta automatización libera aproximadamente 56 minutos de trabajo manual por día, cerca de 19 horas al mes que el equipo

1.	InProgress	-	Aug 07, 2026 09:53:02 PM
2.	InProgress	-	Aug 07, 2026 09:53:01 PM
3.	InProgress	-	Aug 07, 2026 09:53:00 PM
4.	InProgress	-	Aug 07, 2026 09:52:59 PM
5.	InProgress	-	Aug 07, 2026 09:52:57 PM
6.	InProgress	-	Aug 07, 2026 09:52:56 PM
7.	InProgress	-	Aug 07, 2026 09:52:55 PM
8.	InProgress	-	Aug 07, 2026 09:52:54 PM
9.	InProgress	-	Aug 07, 2026 09:52:53 PM
10.	InProgress	-	Aug 07, 2026 09:52:52 PM
11.	InProgress	-	Aug 07, 2026 09:52:51 PM
12.	InProgress	-	Aug 07, 2026 09:52:50 PM
13.	InProgress	-	Aug 07, 2026 09:52:49 PM
14.	InProgress	-	Aug 07, 2026 09:52:48 PM
15.	InProgress	-	Aug 07, 2026 09:52:47 PM
16.	InProgress	-	Aug 07, 2026 09:52:46 PM
17.	InProgress	-	Aug 07, 2026 09:52:44 PM
18.	InProgress	-	Aug 07, 2026 09:52:43 PM
19.	InProgress	-	Aug 07, 2026 09:52:42 PM
20.	InProgress	-	Aug 07, 2026 09:52:41 PM
21.	FatalError	-	Aug 07, 2026 09:51:13 PM
22.	FatalError	-	Aug 07, 2026 09:51:10 PM
23.	CompletedWithErrors	-	Aug 07, 2026 09:50:48 PM
24.	CompletedSuccessfully	-	Aug 07, 2026 09:50:48 PM
25.	CompletedSuccessfully	-	Aug 07, 2026 09:50:48 PM

Figura 10: Los cuatro estados detectados por el bot de monitoreo de TIBCO Scribe.

puede destinar a la atención de incidentes reales en lugar de revisiones repetitivas.

En el caso de AWS, Google Cloud Platform y ServiceNow, el objetivo de verificar y reportar alertas también se cumplió, pero de forma manual en los tres casos; a diferencia de TIBCO Scribe, no se llegó a desarrollar una automatización similar para ellas, principalmente porque el tiempo de la Estadía se concentró en la herramienta que representaba la carga de trabajo más repetitiva. Esa automatización pendiente para AWS, GCP y ServiceNow queda como una oportunidad de mejora a futuro.

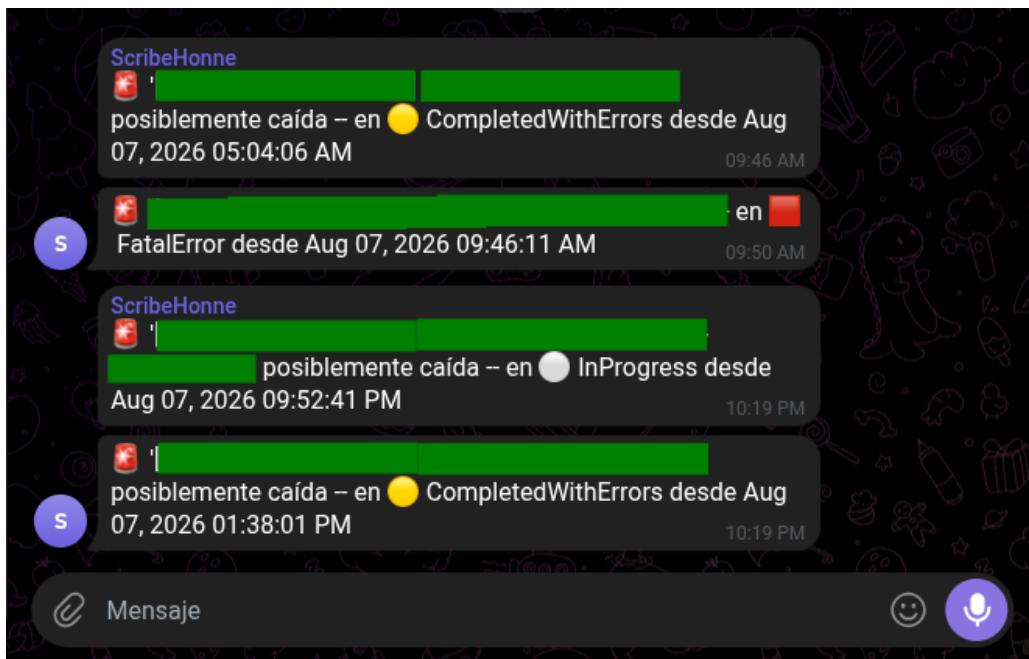


Figura 11: Aviso real enviado por el bot al grupo de Telegram del equipo.

7. Conclusiones y recomendaciones

7.1. Cierre por objetivo específico

- **Verificar las alertas activas de AWS.** Se cumplió: las 129 alertas registradas en el diagnóstico inicial se clasificaron por completo entre patrones programados y fallas reales.
- **Revisar los logs de instancias en Google Cloud Platform.** Se cumplió, aplicando los criterios de más de 20 registros de error y el umbral de persistencia de latencia descritos en el Capítulo 4, de forma manual durante toda la Estadía.
- **Monitorear el estado de las soluciones y Agents de TIBCO Scribe.** Se cumplió, inicialmente de forma manual y después apoyado por el script de automatización.
- **Desarrollar un script de monitoreo automatizado para TIBCO Scribe.** Se cumplió y se superó lo planteado: la versión final del script automatizó incluso la regla del estado de error fatal, que en un inicio se pensaba mantener manual, y se agregó la posibilidad de silenciar alertas conocidas.
- **Dar seguimiento a las alertas de ServiceNow.** Se cumplió de forma manual y constante a lo largo de toda la Estadía, ya que esta herramienta no generaba notificaciones automáticas.

7.2. Aprendizaje profesional y personal

A nivel profesional, la Estadía permitió profundizar en el funcionamiento de distintos servicios en la nube (Software as a Service) y en cómo se procesan las peticiones entre un cliente y estos servicios, así como comprender de forma práctica la importancia de automatizar procesos de monitoreo repetitivos, no solo para ahorrar tiempo sino para reducir el margen de error humano al revisar la misma información una y otra vez. También se reforzó la capacidad de distinguir una falla real de un falso positivo, un criterio que se desarrolla más con la práctica que con la teoría.

A nivel personal, el mayor reto al inicio fue la comunicación con el personal del área: acostumbrarme a consultar dudas, coordinar tareas y reportar hallazgos con compañeros y superiores que ya tenían experiencia en el equipo. Con el tiempo esto dejó de ser un obstáculo. En cuanto al conocimiento técnico, llegué con las bases de AWS y salí con esas mismas bases más reforza-

das por la práctica, ya que el trabajo en la Mesa de Servicio se concentró en el monitoreo con CloudWatch y no hubo oportunidad de profundizar en otras áreas de la plataforma. Lo que más me deja satisfecho de la Estadía es haber logrado que el script de monitoreo de TIBCO Scribe funcionara correctamente, al ser un desarrollo que hice por iniciativa propia.

7.3. Recomendaciones a la empresa

La principal recomendación es extender la automatización de monitoreo a las demás herramientas del área. El caso de TIBCO Scribe mostró que automatizar una revisión repetitiva libera tiempo del equipo para atender incidentes reales; aplicar un enfoque similar a la verificación de alertas de AWS, los logs de Google Cloud Platform y, sobre todo, a ServiceNow —donde actualmente no existe ningún tipo de notificación automática y se depende de la vigilancia constante del equipo— podría traer el mismo beneficio.

7.4. Recomendaciones al programa académico

Se recomienda al programa académico reforzar contenidos prácticos relacionados con plataformas de nube pública (AWS, Google Cloud Platform) y conceptos de monitoreo de infraestructura, ya que durante la Estadía este conocimiento tuvo que adquirirse directamente en campo, sin una base previa específica en la formación académica más allá de la materia de Redes.

7.5. Trabajo pendiente o siguiente fase

Como trabajo pendiente para una siguiente fase, se identifican dos líneas principales: extender la automatización desarrollada para TIBCO Scribe a las demás herramientas de monitoreo (AWS, Google Cloud Platform y ServiceNow), y migrar el script de monitoreo de TIBCO Scribe desde mi equipo de cómputo personal hacia un servidor o equipo de la empresa, para que continúe funcionando de forma permanente una vez concluida la Estadía.

Referencias

- [1] Honne S.A. de C.V. *Transformación Tecnológica y Cloud Computing*. 2026. URL: <https://honne.com/> (visitado 17-06-2026).
- [2] ManageEngine. *Dominando el ruido: una guía para solucionar los falsos positivos de las alertas SIEM*. URL: <https://www.manageengine.com/latam/log-management/siem/reducir-alertas-de-falsos-positivos-siem.html> (visitado 07-08-2026).
- [3] ManageEngine. *Proceso de gestión de incidentes mayores según ITIL con ejemplos reales*. URL: <https://www.manageengine.com/latam/service-desk/itsm/ejemplos-de-mayores-incidentes-til.html> (visitado 07-08-2026).
- [4] ToBeIT. *Automatización en Monitorización y Observabilidad*. URL: <https://tobeit.es/en/automatizacion-en-monitorizacion-y-observabilidad/> (visitado 07-08-2026).
- [5] Daniel Costa. *Ingeniería Inversa de APIs: Guía, Herramientas y Técnicas*. 14 de ene. de 2026. URL: <https://apidog.com/es/blog/reverse-engineering-apis-2/> (visitado 08-08-2026).

Anexos

Cronograma de actividades

- **Primer mes (mediados de mayo – mediados de junio):** verificación de alertas activas de AWS.
- **Segundo mes (mediados de junio – mediados de julio):** revisión de logs de instancias en Google Cloud Platform.
- **Tercer mes en adelante (mediados de julio – cierre de la Estadía):** monitoreo de TIBCO Scribe y desarrollo del script de automatización.
- **Constante durante toda la Estadía:** seguimiento a las alertas de ServiceNow.

Código fuente del script de monitoreo

A continuación se incluye el código completo del script de monitoreo de TIBCO Scribe desarrollado durante la Estadía, descrito en el Capítulo 4.

Listing 2: Script completo de monitoreo de TIBCO Scribe.

```
1 from dotenv import load_dotenv
2 load_dotenv()
3
4 import os
5 import json
6 import base64
7 import time
8 import threading
9 import requests
10 from datetime import datetime, timedelta, timezone
11 from zoneinfo import ZoneInfo
12
13 ZONA_LOCAL = ZoneInfo("America/Mexico_City")
14
15 TELEGRAM_TOKEN = os.environ["TELEGRAM_TOKEN"]
16 TELEGRAM_CHAT_ID = os.environ["TELEGRAM_CHAT_ID"]
17
```



```
18 estado_lock = threading.Lock()
19
20
21 def enviar_telegram(mensaje):
22     url = f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/sendMessage"
23     try:
24         resp = requests.post(url, data={"chat_id": TELEGRAM_CHAT_ID, "text": mensaje})
25         resp.raise_for_status()
26         return resp.json()["result"]["message_id"]
27     except Exception as e:
28         print(f"!!No se pudo enviar a Telegram: {e}")
29         return None
30
31
32 def borrar_mensaje_telegram(message_id):
33     url = f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/deleteMessage"
34     try:
35         resp = requests.post(url, data={"chat_id": TELEGRAM_CHAT_ID, "message_id":
36             message_id})
37         resp.raise_for_status()
38     except Exception as e:
39         print(f"!!No se pudo borrar el mensaje de Telegram: {e}")
40
41
42 def avisar_inicio_bot():
43     message_id = enviar_telegram("_Bot de monitoreo iniciado")
44     if message_id is not None:
45         time.sleep(2)
46         borrar_mensaje_telegram(message_id)
47
48 def formatear_hora_local(dt_utc):
49     return dt_utc.astimezone(ZONA_LOCAL).strftime("%b %d, %Y %I: %M: %S %p")
50
51
52 MUTE_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "muted.json")
53 MUTE_DURACION_MINUTOS = 60
```

```
54
55 muted_soluciones = {}
56
57 soluciones_conocidas = {}
58
59
60 def cargar_mutes():
61     global muted_soluciones
62     if not os.path.exists(MUTE_FILE):
63         return
64
65     try:
66         with open(MUTE_FILE, "r", encoding="utf-8") as f:
67             data = json.load(f)
68     except Exception as e:
69         print(f"¡No se pudo leer {MUTE_FILE}: {e}")
70         return
71
72     ahora = datetime.now(timezone.utc)
73     vigentes = {}
74     for codigo, expiracion_iso in data.get("codigos", {}).items():
75         try:
76             expiracion = datetime.fromisoformat(expiracion_iso)
77         except Exception:
78             continue
79         if expiracion > ahora:
80             vigentes[codigo] = expiracion
81
82     with estado_lock:
83         muted_soluciones = vigentes
84
85     if vigentes:
86         print(f"Mutes cargados desde disco: {list(vigentes.keys())}")
87
88
89 def guardar_mutes():
90     data = {
```

```

91     "codigos": {
92         codigo: expiration.isoformat()
93         for codigo, expiration in sorted(muted_soluciones.items())
94     }
95 }
96 try:
97     with open(MUTE_FILE, "w", encoding="utf-8") as f:
98         json.dump(data, f, indent=2, ensure_ascii=False)
99 except Exception as e:
100     print(f"!!!No se pudo guardar {MUTE_FILE}: {e}")
101
102
103 def limpiar_mutes_vencidos():
104     ahora = datetime.now(timezone.utc)
105     vencidos = [codigo for codigo, expiration in muted_soluciones.items() if
106                 expiration <= ahora]
107     for codigo in vencidos:
108         del muted_soluciones[codigo]
109     if vencidos:
110         guardar_mutes()
111
112 def es_muteada(nombre_solucion):
113     nombre_upper = nombre_solucion.strip().upper()
114     with estado_lock:
115         limpiar_mutes_vencidos()
116         return any(nombre_upper.startswith(codigo) for codigo in muted_soluciones)
117
118
119 def buscar_nombres_por_codigo(codigo):
120     resultados = []
121     with estado_lock:
122         for org_nombre, nombre in soluciones_conocidas.values():
123             if nombre.strip().upper().startswith(codigo):
124                 resultados.append((org_nombre, nombre))
125     return resultados
126

```

```

127
128 SCRIBESOFT_USER = os.environ["SCRIBESOFT_USER"]
129 SCRIBESOFT_PASS = os.environ["SCRIBESOFT_PASS"]
130
131 credenciales = base64.b64encode(f"{SCRIBESOFT_USER}:{SCRIBESOFT_PASS}".encode("utf-8"
132     )),.decode("ascii")
133
134 scribesoft_session = requests.Session()
135 scribesoft_session.headers["Authorization"] = f"Basic_{credenciales}"
136
137 ORGANIZACIONES = [
138     {"id": "CLIENTE_1_ID", "nombre": "CLIENTE_1"},
139     {"id": "CLIENTE_2_ID", "nombre": "CLIENTE_2"},
140 ]
141
142 LIMIT_HISTORIAL = 25
143 INTERVALO_SCRIBESOFT_SEGUNDOS = 240
144
145 ESTADO_OK = "CompletedSuccessfully"
146
147 REGLAS = {
148     "InProgress": {"tipo": "conteo_y_tiempo", "limite": 10, "limite_min": 10},
149     "CompletedWithErrors": {"tipo": "conteo", "limite": 10},
150 }
151
152 COLORES = {
153     "CompletedSuccessfully": "CompletedSuccessfully",
154     "InProgress": "InProgress",
155     "CompletedWithErrors": "CompletedWithErrors",
156     "FatalError": "FatalError",
157 }
158
159 rachas_activas = {}
160
161 fatal_errors_reportados = {}
162 FATAL_ERROR_REPORT_TTL_MINUTOS = 60 * 24

```

```
163 fatal_error_primera_pasada = set()
164
165
166 def obtener_soluciones(org_id):
167     resp = scribesoft_session.get(f"https://api.scribesoft.com/v1/orgs/{org_id}/
168         solutions")
169     resp.raise_for_status()
170     return resp.json()
171
172 def obtener_historial(org_id, sol_id):
173     resp = scribesoft_session.get(
174         f"https://api.scribesoft.com/v1/orgs/{org_id}/solutions/{sol_id}/history",
175         params={
176             "offset": 0,
177             "limit": LIMIT_HISTORIAL,
178             "executionHistoryColumnSort": 0,
179             "laterThanDate": "1753-01-01",
180             "sortOrder": "desc",
181         },
182     )
183     resp.raise_for_status()
184     return resp.json()
185
186
187 def calcular_racha(historial):
188     if not historial:
189         return None, None
190
191     estado_actual = historial[0]["result"]
192
193     if estado_actual == ESTADO_OK:
194         return None, None
195
196     racha = []
197     for item in historial:
198         if item["result"] == estado_actual:
```

```

199         racha.append(item)
200     else:
201         break
202
203     mas_viejo_de_la_racha = racha[-1]
204     inicio = datetime.fromisoformat(mas_viejo_de_la_racha["start"].replace("Z", "
205         +00:00"))
206
207     return estado_actual, inicio
208
209 def limpiar_fatal_errors_vencidos():
210     limite = datetime.now(timezone.utc) - timedelta(minutes=
211         FATAL_ERROR_REPORT_TTL_MINUTOS)
212     vencidos = [k for k, momento in fatal_errors_reportados.items() if momento <
213         limite]
214     for k in vencidos:
215         del fatal_errors_reportados[k]
216
217 def reportar_fatal_errors_del_historial(org_nombre, nombre, clave, historial):
218     limpiar_fatal_errors_vencidos()
219
220     es_primera_pasada = clave not in fatal_error_primera_pasada
221     fatal_error_primera_pasada.add(clave)
222
223     for item in historial:
224         if item["result"] != "FatalError":
225             continue
226
227         clave_evento = f"{clave}|{item['start']}"
228         if clave_evento in fatal_errors_reportados:
229             continue
230
231         fatal_errors_reportados[clave_evento] = datetime.now(timezone.utc)
232
233         if es_primera_pasada:

```

```
233         continue
234
235     if es_muteada(nombre):
236         continue
237
238     inicio = datetime.fromisoformat(item["start"].replace("Z", "+00:00"))
239     reportar_scribesoft(f"[{org_nombre}]_{nombre}", "FatalError", inicio)
240
241
242 def umbral_para_empezar_a_vigilar(estado):
243     regla = REGLAS.get(estado)
244     if regla is None:
245         return None
246
247     if "limite" in regla:
248         return regla["limite"]
249
250     return 1
251
252
253 def contar_racha(historial, estado):
254     cantidad = 0
255     for item in historial:
256         if item["result"] == estado:
257             cantidad += 1
258         else:
259             break
260     return cantidad
261
262
263 def debe_reportar(estado, inicio, cantidad_en_racha):
264     regla = REGLAS.get(estado)
265
266     if regla is None:
267         return False
268
269     if regla["tipo"] == "conteo":
```

```

270     return cantidad_en_racha >= regla["limite"]
271
272     if regla["tipo"] == "tiempo":
273         ahora = datetime.now(timezone.utc)
274         minutos_transcurridos = (ahora - inicio).total_seconds() / 60
275         return minutos_transcurridos >= regla["limite_min"]
276
277     if regla["tipo"] == "conteo_y_tiempo":
278         ahora = datetime.now(timezone.utc)
279         minutos_transcurridos = (ahora - inicio).total_seconds() / 60
280         cumple_conteo = cantidad_en_racha >= regla["limite"]
281         cumple_tiempo = minutos_transcurridos >= regla["limite_min"]
282         return cumple_conteo and cumple_tiempo
283
284     return False
285
286
287 def reportar_scribesoft(nombre, estado, inicio):
288     color = COLORES.get(estado, f"_{estado}")
289     mensaje = f"_{nombre}'_posiblemente_cada_{en_{color}}_desde_{
290         formatear_hora_local(inicio)}"
291     print(f"\n{mensaje}\n")
292     enviar_telegram(mensaje)
293
294 def revisar_scribesoft():
295     print(f"\n===_Revisin_Scribesoft:_{datetime.now(ZONA_LOCAL).strftime('%b_%d,%_Y_%
296         I: %M: %S_%p')}_===")
297
298     resumen_rachas = []
299
300     for org in ORGANIZACIONES:
301         org_id = org["id"]
302         org_nombre = org["nombre"]
303
304         print(f"\n#####_Organizacin:_{org_nombre}_#####")

```



```

305     soluciones = obtener_soluciones(org_id)
306
307     for sol in soluciones:
308         nombre = sol.get("name", "(sin_nombre)")
309         sol_id = sol["id"]
310         clave = f"{org_id}:{sol_id}"
311
312         with estado_lock:
313             soluciones_conocidas[clave] = (org_nombre, nombre)
314
315         print(f"\n[{org_nombre}]_/{nombre}")
316
317         historial = obtener_historial(org_id, sol_id)
318
319         if not historial:
320             print("_(_(sin_historial)")
321             continue
322
323         for j, item in enumerate(historial, start=1):
324             estado_item = item["result"]
325             color_item = COLORES.get(estado_item, f"_SIN_MAPEAR_{estado_item}")
326             dt_item = datetime.fromisoformat(item["start"].replace("Z", "+00:00"))
327             print(f"_{j}._{color_item}_{estado_item}_{formatear_hora_local(dt_item)}")
328
329         reportar_fatal_errors_del_historial(org_nombre, nombre, clave, historial)
330
331         estado_actual, inicio_racha = calcular_racha(historial)
332
333         if estado_actual is None:
334             if clave in rachas_activas:
335                 del rachas_activas[clave]
336                 continue
337
338         cantidad = contar_racha(historial, estado_actual)
339         umbral = umbral_para_empezar_a_vigilar(estado_actual)
340
341         if umbral is None or cantidad < umbral:

```

```

342         if clave in rachas_activas:
343             del rachas_activas[clave]
344         continue
345
346         if clave not in rachas_activas or rachas_activas[clave]["inicio"] !=
           inicio_racha:
347             rachas_activas[clave] = {"estado": estado_actual, "inicio":
           inicio_racha, "reportado": False}
348
349         info = rachas_activas[clave]
350         color = COLORES.get(estado_actual, f"_{estado_actual}")
351         muteada = es_muteada(nombre)
352         tag_mute = "_(muteada, no se manda alerta)" if muteada else ""
353         resumen_rachas.append(
354             f"_{org_nombre}_{nombre}_{color}_{cantidad}_{seguidos}_{desde}_{
           formatear_hora_local(info['inicio'])}_{tag_mute}"
355         )
356
357         if not muteada and not info["reportado"] and debe_reportar(estado_actual,
           info["inicio"], cantidad):
358             reportar_scribesoft(f"_{org_nombre}_{nombre}", estado_actual, info["
           inicio"])
359             info["reportado"] = True
360
361         print("\n" + "=" * 60)
362         print("RESUMEN_DE_EJECUCIONES_ACTIVAS")
363         if resumen_rachas:
364             for linea in resumen_rachas:
365                 print(linea)
366         else:
367             print("_(ninguna, todo en orden)")
368
369
370 def loop_scribesoft():
371     while True:
372         try:
373             revisar_scribesoft()

```

```

374     except Exception as e:
375         print(f"!!_Error_durante_la_revisin_de_Scribesoft:_{e}")
376         time.sleep(INTERVALO_SCRIBESOFT_SEGUNDOS)
377
378
379 def procesar_comando(texto, responder):
380     partes = texto.strip().split(maxsplit=1)
381     comando = partes[0].lower()
382
383     if comando == "/mute":
384         if len(partes) < 2 or not partes[1].strip():
385             responder("Uso:_/mute_<codigo>_[horas]_ (ej._/mute_008B_o_/_mute_008B_3)")
386             return
387
388         codigo_y_resto = partes[1].strip().split(maxsplit=1)
389         codigo = codigo_y_resto[0].strip().upper()
390
391         if len(codigo_y_resto) > 1:
392             try:
393                 horas = float(codigo_y_resto[1].strip().replace(",", "."))
394                 if horas <= 0:
395                     raise ValueError
396             except ValueError:
397                 responder("Nmero_de_horas_invlido._Uso:_/mute_<codigo>_[horas]_ (ej._/
398                     mute_008B_3)")
399                 return
400
401                 duracion_minutos = horas * 60
402
403             else:
404                 duracion_minutos = MUTE_DURACION_MINUTOS
405
406             expiracion = datetime.now(timezone.utc) + timedelta(minutes=duracion_minutos)
407
408             with estado_lock:
409                 muted_soluciones[codigo] = expiracion
410                 guardar_mutes()
411
412             hora_local = formatear_hora_local(expiracion)

```

```

410     matches = buscar_nombres_por_codigo(codigo)
411
412     mensaje = f"_{codigo}',_hasta_las_{hora_local}."
413     if matches:
414         mensaje += "\nSolutions_que_matchean_ahora_mismo:\n" + "\n".join(
415             f"_{org}]_{nombre}" for org, nombre in matches
416         )
417     else:
418         mensaje += "\n(No_encontr_ninguna_solution_con_ese_codigo_todava;_se_aplicar
419             _en_cuanto_aparezca_en_una_revisin.)"
420
421     responder(mensaje)
422
423 elif comando == "/unmute":
424     if len(partes) < 2 or not partes[1].strip():
425         responder("Uso:_/unmute_<codigo>_(ej._/unmute_008B)")
426         return
427
428     codigo = partes[1].strip().upper()
429
430     with estado_lock:
431         existia = codigo in muted_soluciones
432         if existia:
433             del muted_soluciones[codigo]
434             guardar_mutes()
435
436         if existia:
437             responder(f"_{codigo}',_ya_no_est_muteado.")
438         else:
439             responder(f"_{codigo}',_no_estaba_muteado.")
440
441 elif comando == "/muted":
442     with estado_lock:
443         limpiar_mutes_vencidos()
444         copia = dict(muted_soluciones)
445
446     if not copia:

```

```

446         responder("No hay mutes activos.")
447         return
448
449         lineas = ["Mutes activos:"]
450         for codigo, expiracion in sorted(copia.items()):
451             hora_local = formatear_hora_local(expiracion)
452             matches = buscar_nombres_por_codigo(codigo)
453             nombres = ", ".join(nombre for _, nombre in matches) if matches else "sin match todava"
454             lineas.append(f"{codigo} (hasta {hora_local}) -> {nombres}")
455
456         responder("\n".join(lineas))
457
458     elif comando.startswith("/"):
459         responder(
460             "Comandos disponibles:\n"
461             "/mute<codigo>[horas] [silencia alertas de esa solution (1 hora por default, ej. /mute 008B3)]\n"
462             "/unmute<codigo> [quita el mute antes de que expire]\n"
463             "/muted [lista los mutes activos]"
464         )
465
466
467 TELEGRAM_POLL_BACKOFF_INICIAL_SEGUNDOS = 5
468 TELEGRAM_POLL_BACKOFF_MAXIMO_SEGUNDOS = 60
469
470
471 def escuchar_comandos_telegram():
472     url = f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/getUpdates"
473     offset = None
474     backoff = TELEGRAM_POLL_BACKOFF_INICIAL_SEGUNDOS
475
476     poll_session = requests.Session()
477
478     print("Escuchando comandos de Telegram (/mute, /unmute, /muted)...")
479
480     while True:

```

```

481     try:
482         params = {"timeout": 30}
483         if offset is not None:
484             params["offset"] = offset
485
486         resp = poll_session.get(url, params=params, timeout=35)
487         resp.raise_for_status()
488         data = resp.json()
489
490         backoff = TELEGRAM_POLL_BACKOFF_INICIAL_SEGUNDOS
491
492         for update in data.get("result", []):
493             offset = update["update_id"] + 1
494
495             mensaje = update.get("message")
496             if not mensaje or "text" not in mensaje:
497                 continue
498
499             chat_id = mensaje.get("chat", {}).get("id")
500             if str(chat_id) != str(TELEGRAM_CHAT_ID):
501                 continue
502
503             texto = mensaje["text"]
504             if not texto.startswith("/"):
505                 continue
506
507             print(f"_{chat_id}<_comando_recibido:_{texto}")
508             procesar_comando(texto, enviar_telegram)
509
510         except (requests.exceptions.ConnectionError, requests.exceptions.Timeout,
511               requests.exceptions.ChunkedEncodingError) as e:
512             print(f"!!_Error_de_red_escuchando_comandos_de_Telegram_(reintentando_en_{
513                   backoff}s):_{e}")
514             poll_session.close()
515             poll_session = requests.Session()
516             time.sleep(backoff)
517             backoff = min(backoff * 2, TELEGRAM_POLL_BACKOFF_MAXIMO_SEGUNDOS)

```

```
516     except Exception as e:
517         print(f"!!_Error_escuchando_comandos_de_telegram_(reintentando_en_{backoff}
518             s):_{e}")
519         time.sleep(backoff)
520         backoff = min(backoff * 2, TELEGRAM_POLL_BACKOFF_MAXIMO_SEGUNDOS)
521
522 def main():
523     print("Bot_de_monitoreo_iniciado._Ctrl+C_para_detener._\n")
524
525     cargar_mutes()
526     avisar_inicio_bot()
527
528     threading.Thread(target=loop_scribesoft, daemon=True).start()
529
530     escuchar_comandos_telegram()
531
532
533 if __name__ == "__main__":
534     main()
```

Instituto Mexicano del Seguro Social

Constancia de Vigencia de Derechos

Homoclave del trámite	Homoclave del formato	Fecha de publicación del formato en el DOF
IMSS-02-020	FF-IMSS-012	10 / 11 / 2015 DD MM AAAA

Datos Generales

NSS:	38160153201
CURP:	ZUZE010524HTSXVDA8
Nombre(s), primer apellido y segundo apellido:	EDER OMAR ZU#IGA ZAVALA
Sexo:	Hombre
Fecha de nacimiento:	24/05/2001
Lugar de nacimiento:	TAMAULIPAS

Datos de Aseguramiento

Con derecho al servicio médico:	SI
Vigente:	06/02/2026
Delegación:	TAMAULIPAS
UMF:	UMF 067 SAN LUISITO
Turno:	VESPERTINO
Consultorio:	CONSULTORIO 11
Agregado Médico:	1M2001ES

Datos de Aseguramiento

Registro Patronal	Nombre o razón social
F0543370327	UNIVERSIDAD POLITECNICA DE VICTORIA
Modalidad de Aseguramiento	Descripción de Modalidad
MODALIDAD 32	SEGURO FACULTATIVO ESTUDIANTES

Detalle de vigencia

Estado	Inicio de Vigencia	Fecha de Constancia
VIGENTE	03/02/2026	06/02/2026

Beneficiarios

NO APLICA

"De conformidad con los artículos 4 y 69-M, fracción V de la Ley Federal de Procedimiento Administrativo, los formatos para solicitar trámites y servicios deberán publicarse en el Diario Oficial de la Federación (DOF)"

**GOBIERNO DE
MÉXICO****Contacto**

Paseo de la Reforma 476, P.B.
Col. Juárez, Alcaldía Cuauhtémoc
C.P. 06600, Ciudad de México.
Tel. 800 623 23 23
<http://www.imss.gob.mx/contacto>

Ciudad Victoria,
Tamaulipas, a 01
de Mayo
de 2026

HONNE - MULTICLOUD EXPERIENCE
OSCAR GONZALEZ GONZALEZ
PRESENTE

La Universidad Politécnica de Victoria tiene a bien presentar a **ZUÑIGA ZAVALA EDER OMAR** estudiante del programa académico de **INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL** en su modalidad de **DESARROLLO DE SOFTWARE MULTIPLATAFORMA**, con número de matrícula **2430206** y seguro facultativo IMSS número **38160153201**; quien deberá realizar su práctica profesional de **ESTADÍA TSU**, a partir del 04 de Mayo de 2026 al 14 de Agosto de 2026, con duración de **600 horas** desarrollando el proyecto **"HONNE INTELLIGENCE HUB"** que le fue asignado.

Al concluir se le extenderá la carta de liberación al evaluarlo satisfactoriamente y además le solicitaremos su valiosa opinión respondiendo el formulario que se le enviará por correo electrónico, referente al desempeño del practicante, la información que nos proporcione es de vital importancia para la mejora de los programas académicos que ofrece la Universidad Politécnica de Victoria para la formación de profesionistas altamente especializados.

El practicante deberá cumplir con el Reglamento Interno aplicable al personal en su centro de trabajo.

Sin otro particular.

ATENTAMENTE



OTHÓN CANO GARZA
DIRECTOR DE VINCULACIÓN



ALICIA GUADALUPE DIAZ SANCHEZ
Gerente de Gestión de Talento
Grupo Honnexs

UNIVERSIDAD POLITÉCNICA DE VICTORIA

Av. Nuevas Tecnologías 5902
Parque Científico y Tecnológico de Tamaulipas
Carretera Victoria-Soto La Marina Km. 5.5
Cd. Victoria, Tamaulipas. C.P. 87138

Tel: (834) 1711000 al 100
www.upvictoria.edu.mx



Cd. Victoria, Tam a 30 de abril de 2026.

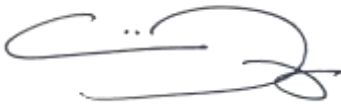
MTRO. OTHON CANO GARZA.
DIRECTOR DE VINCULACION.
UPV VICTORIA.
PRESENTE.-

ASUNTO: ACEPTACION DE PRACTICAS

Por medio de la presente le comunico que **ZUÑIGA ZAVALA EDER OMAR** alumno de la carrera de **INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL** ha sido aceptado en esta Institución a fin de que realice sus prácticas profesionales dentro del *PROYECTO "HONNE INTELLIGENCE HUB"* en el área de *OPERACIONES – Mesa de Servicio y Monitoreo*, a partir del **04 de mayo de 2026**, en un horario de 09.00 am a 03.00 pm de lunes a viernes.

Se extiende la presente para los fines que así considere convenientes.

ATENTAMENTE,



Alicia Guadalupe Díaz Sánchez

Gerente de Gestión de Talento

Grupo Honnexs

alicia.diaz@honne.com

ccp. Expediente Practicante.

Rúbrica para la Evaluación del Reporte de Estadía

Nombre completo del estudiante: Eder Omar Zúñiga Zavala Matrícula: 2430206 PERIODO: Mayo-Agosto 2026 Nombre del evaluador: Dr. Said Polanco Martagón			
Firma del Evaluador: _____ Calificación Final: ____ / 100			
Valor	Características a cumplir	Puntos	Observaciones
6	Resumen ejecutivo y abstract: Indica qué se realizó, cómo se realizó y qué se obtuvo. (una cuartilla en español e inglés)		
6	Contenido temático: Describe correctamente el contenido del documento		
6	Introducción: El informe cumple con introducir al lector de una manera atractiva el panorama general del trabajo realizado en la unidad económica.		
6	Descripción de la unidad económica: Se redactan los detalles de los antecedentes del departamento, la empresa, giro, misión, visión, infraestructura, ubicación.		
6	Objetivos operativos: Describen las acciones a cumplir mediante actividades establecidas por el asesor empresarial		
24	Metodología: Descripción a detalle de las etapas y procedimientos utilizados en el proyecto		
18	Resultados: Interpreta y analiza los resultados obtenidos durante su estadía.		
12	Conclusiones: Las conclusiones son claras y acordes con el objetivo esperado		
6	Bibliografía y anexos: Cita correctamente las fuentes de información utilizando un estilo de referencia; anexa cartas correspondientes		
10	Responsabilidad: Entregó el reporte en la fecha y hora señalada. Asistió al menos al 80 % de sus días de Estadía		

Rúbrica para Exposición del Reporte de Estadía

Criterio	Nivel de logro		
	Bueno (10 %)	Regular (7 %)	Menor a lo esperado (5 %)
Dominio del tema (10 %)	Utiliza todos los términos y conceptos de manera correcta.	Utiliza la mayoría términos y conceptos de manera correcta	Confunde los términos y conceptos.
Presentación de material visual (10 %)	Las diapositivas usadas tienen fondo claro, letras, tablas e imágenes visibles. La presentación contiene los aspectos relevantes de la Estadía	Las diapositivas usadas presentan dificultad para su apreciación visual. La presentación contiene los aspectos relevantes de la Estadía	Las diapositivas usadas presentan dificultad para su apreciación visual. La presentación carece de algunos aspectos relevantes de la Estadía.
Comunicación y expresión (10 %)	Su volumen de voz es comprensible. Su expresión corporal manifiesta seguridad ejecutiva. Su atuendo cumple los estándares de ser formal o semiformal. Mantiene una línea de respeto en general.	Su volumen de voz no es comprensible. Su expresión corporal manifiesta inseguridad. Su atuendo cumple los estándares de ser formal o semiformal. Mantiene una línea de respeto en general.	Su volumen de voz no es comprensible. Su expresión corporal manifiesta inseguridad. Su atuendo no cumple los estándares de ser formal o semiformal. Realiza alguna falta de respeto.
Respuestas (10 %)	Las respuestas son consistentes y con parámetros de lógica operativa.	Las respuestas no son consistentes pero tienen parámetros de lógica operativa.	Las respuestas son difusas e incoherentes.
Ponderación final de su exposición:		___ /100	
Día / Mes / Año		12/ Agosto/ 2026	
Comentario adicional			